Habu: A Checked Concatenative Kernel Target and Model CAD for GPUs

Joel Reymont
joelr1@gmail.com

Draft of July 4, 2026

Abstract

Writing custom GPU kernels remains an expert activity in which most errors surface only at runtime, and the artifacts of machine-learning compilation are emitted rather than proven. We present Habu, a self-hosted checked Forth whose compiler verifies a typed, row-polymorphic stack effect for every word, and Maki, a model-development layer that treats GPU compilation as computer-aided design: models are captured into an IR, fused by a planner under typed legality rules with exact byte accounting, lowered to generated PTX kernels—one per fusion region, for elementwise, row-reduce, matmul, and movement region classes—and *verified*: every generated kernel, and the whole multi-region model, is proven element-wise against a checked host reference on the target device before its artifact is promoted with a recorded evidence trail. The same IR differentiates: adjoints are generated, gradient-checked against finite differences, and used to train a model from scratch through the identical pipeline, with a seeded convergence gate committed as a test. The entire toolchain is a single self-hosted binary under 128 KB. On a Jetson Orin NX we measure that this verification carries no measured performance tax: generated, gate-proven kernels match hand-written Triton at the streaming-memory roof, while a whole class of kernel-authoring errors—measured head-to-head against Triton on the same device—moves from runtime to author time, and the gates catch defect classes (silent copy-back loss, malformed generated modules, wrong adjoints) that the status quo ships. The toolchain is four orders of magnitude smaller than the stacks it replaces. **[TODO: Extend with compute-bound results when measured (§8).]**

1 Introduction

The deep learning compilation landscape divides labor in a familiar way: vendor libraries cover common operations; kernel DSLs such as Triton [1] let experts write the rest; graph compilers fuse and schedule above them. Two costs are accepted as normal. First, kernel authorship is unchecked: a Triton author who mis-tracks a mask or an offset discovers it at runtime, if at all. Second, the artifacts of compilation are *emitted, not proven*: nothing in the toolchain certifies that the fused kernel computes what the unfused graph did, that a copy-back actually happened, or that a generated gradient is the gradient.

Habu rejects both costs. It is a Forth in the lineage of Chuck Moore's OKAD [2]: a small, self-hosted system—engine, compiler, and checker fit in one binary under 128 KB—in which programs are compositions of words. The departure from tradition is that every word carries a typed, row-polymorphic stack effect that the self-hosted checker verifies; kernels for the GPU are written, or generated, in the same checked language and emitted as PTX text.

Maki, the model layer, treats the pipeline above kernels as computer-aided design rather than black-box compilation. A model is captured into an IR by a checked definition (`MODEL:`); a fusion

planner groups nodes into regions under typed legality rules, reporting exact traffic in bytes; a memory planner reports coalescing classes; a schedule layer names candidate kernels per region family under a content-addressed key; and a lowering layer generates one PTX kernel per region. At every stage the system emits facts that a person—or an agent—can audit, and the final artifact is admitted only through gates (§5.1). Only a model that passes is *promoted*: its schedule selection and evidence are recorded in a content-addressed store.

This paper contributes:

1. a checked concatenative kernel target that *eliminates* the stack-discipline and composition error class at author time, measured head-to-head against Triton on identical tasks and hardware (§7);
2. Model CAD: the first ML compilation pipeline we know of whose artifacts are admitted by device-proof with recorded evidence rather than emitted on trust—at measured parity with the status quo where memory-bound, i.e., verification without a performance tax (§5);
3. a verified-gradient facility in which the training IR and the gradchecked IR are the same object, demonstrated by training a model from scratch through the pipeline (§6);
4. measurements on Jetson Orin NX (§8). **[TODO: contribution (5): compute-bound performance, once measured.]**

2 Related Work

Tillet et al. [1] classify prior systems as tensor-level IRs (XLA [6], Glow [8]: pattern-matched templates), polyhedral compilers (Tensor Comprehensions [5], Diesel: affine loop nests), and loop synthesizers (Halide [3], TVM [4]: user-specified schedules), positioning Triton as tile-level operations inside a traditional pipeline with compiler-owned parallelization. Habu shares Triton's durable insight—the author writes a scalar-looking program over tiles while the compiler owns intra-tile concurrency—but differs on three axes. First, the composition is *typed*: fused chains are checked words whose stack effects the compiler verifies, so a malformed composition fails to compile rather than misbehaving on device. Second, fusion is a *planner decision with an audit trail* (regions, typed split reasons, byte deltas), not an authoring convention. Third, artifacts are *gated*: device-vs-host verification is part of compilation, not an external test the author may skip.

Automatic fusion above kernel DSLs is the province of graph compilers such as `torch.compile`/Inductor [7], which fuse and emit Triton kernels. Inductor decides fusion but neither types the result nor accounts for it in falsifiable units, and its artifacts carry no verification evidence; Habu's planner reports exact bytes (the same number serves as fusion objective, traffic report, and roofline denominator) and its artifacts are admitted by proof. Modern Triton (a Python DSL on MLIR) retains per-kernel horizons and per-process JIT autotuning; Habu's tuning is persistent and content-keyed, which matters on embedded targets where warmup is a deployment cost. A separate lineage—translation validation and proof-carrying artifacts—informs the gate design: not a formal semantics proof, but a machine-checked, reproducible equivalence test on the target device, recorded per artifact.

3 Habu: The Checked Language

Habu is a concatenative language: a program is a sequence of words, each transforming an implicit stack. The engine is self-hosted—the compiler, checker, and PTX emitters are themselves Habu

programs—and rebuilds itself to a byte-for-byte fixpoint as its release gate; a hash pin over the boot prefix makes mid-build source drift a build failure rather than a silent inconsistency.

Typed stack effects. Every definition carries a signature that is *parsed and enforced*, not a comment:

```
: BW-ACCUM ( n n -- ) {: nc:n ref:n :}
   ref BW-GET {: cur:n :}
   cur BW-NONE = if nc ref BW-SET exit then
   cur nc OP-ADD ref BW-OP2  ref BW-SET ;
```

The `( n n -- )` clause is a type signature over stack rows; the checker verifies that the body's net effect matches, that branch joins agree, and that locals (`{: nc:n ref:n :}`) are used at their declared types. Effects are row-polymorphic, so words compose over arbitrary stack prefixes: a word with effect $(\rho\ \alpha_1 \dots \alpha_m \to \rho\ \beta_1 \dots \beta_n)$ is typable against any stack whose suffix matches $\alpha_1 \dots \alpha_m$, with the row variable ρ binding the untouched prefix. Composition is sequencing—the output row of one word unifies with the input row of the next—and a conditional's arms must yield *unifiable* effects at the join, so a branch that leaks or eats an extra cell is a compile error at the `then`. Quotations carry their effect as a first-class type, so higher-order words (`?do`, combinators, test harnesses) type their bodies. Beyond base shapes (`n`, floats, pointers, quotations), the system uses *nominal roles*: byte-lengths, return codes, and—in the GPU driver—distinct handle types for device, context, module, function, and device pointer, so a context is not passable where a module handle is expected even though both are cell-sized.

Scope and soundness. We claim engineering-grade checking, not a proven-sound calculus: the judgment is enforced by a self-hosted checker that is itself under progressive self-typing, and its residual trust surface is the enumerated `TRUSTED:` set. Typed stack languages are prior art—Cat [9] and Kitten explored typed concatenative calculi, and Factor ships a stack-effect checker—but to our knowledge none types *code-generating kernel compositions* where the checked property is the legality of the emitted GPU program's construction. A formal treatment, together with the algebraic data types now being added to the checker, is future work we intend to report separately.

Fail-closed discipline. Fallible words return error unions or throw *named* codes from documented per-subsystem ranges; silent fallbacks and error masking are prohibited by convention and enforced by lint gates. Where the checker cannot yet express a boundary (a raw syscall pointer walk, an FFI cast), the boundary is a named `TRUSTED:` word with a manifest row and a ratchet: the count of trusted sites may only decrease without explicit review.

Scale. The measured binary is 115,831 bytes on macOS/ARM64 and 102,592 bytes on Linux/AArch64—both committed size-ratchet baselines; growing either requires a deliberate same-commit bump. The full native gate (fixpoint rebuild, engine suite, checker, REPL, AOT build) runs in about seven seconds on a laptop and eleven on the Jetson, which makes "prove everything on every commit" a practical policy.

4 The Checked Kernel Target

GPU kernels are Habu words that *emit* PTX. Each per-operation emitter appends PTX lines and returns the register holding its result, so a fused chain is ordinary composition: the checker types

the emitter sequence, and intermediates are register-resident by construction. A complete fused kernel from the evaluation suite:

```
K ( span<...,extent-n> span<...,extent-n> uniform<f32> -- )
  {: x y a :} x GRID-CTX-V4 {: g :}
  x g LOAD-V4  a SCALE-V4  y g LOAD-V4  ADD-V4  RELU-V4
  y g STORE-V4 ;
```

Two loads, one store, no global round-trips; the type system proves the composition or rejects it with a located diagnostic before any GPU is involved.

Modules. A PTX module is exactly one `.version/.target/.address_size` header followed by N entries. The emitter enforces this with an explicit pair (`PTX-MODULE{ … }PTX-MODULE`); a text regression asserts *exactly one* `.version` per emitted stream, because the bug it guards—two concatenated headers—assembles nowhere yet is invisible to presence-only pattern tests. We found exactly this bug on device; the host gate could not see it because macOS has no `ptxas`.

Fail-closed device proofs. Every host buffer that receives a device copy-back is pre-poisoned with a sentinel pattern and guarded on read, so a silently dropped copy cannot masquerade as a passing golden. Kernel emission is spawned into a child engine whose exit status and stderr are checked and surfaced—a candidate kernel under evaluation is untrusted source, and a crashing candidate must be a graded failure, not a harness casualty. The CUDA driver binding is a typed library layer (nominal handle roles, named-throw return-code guards); no tool in the tree resolves driver symbols by hand.

5 Model CAD

Maki treats model compilation as computer-aided design: every stage produces inspectable facts, every decision has a reason, and the artifact is admitted by proof.

Capture. `MODEL:` parses a signature of named, shaped inputs and a body of operation words into an IR node table:

```
MODEL: FFN-SKIP ( x:4x8 w1:8x16 b1:1x16 w2:16x8 b2:1x8 -- y )
  LINEAR GELU LINEAR x RESIDUAL-ADD RMSNORM ;
```

Shapes like `4x8` are load-time facts validated by capture-time legality: a binary elementwise parameter must equal its data operand's shape or one of the op's documented broadcast classes (bias $1\times C$; scale 1×1 or same-shape—exactly the classes the generated adjoints assume); matmul inner dimensions and linear bias widths are checked; violations throw named errors at load time. Bodies support *named value references*: a signature name or a `>V`-bound intermediate can be pushed as a later operand, so true skip connections and fan-out DAGs are expressible—in the listing above, `x RESIDUAL-ADD` reads input 0, and the backward pass will sum both cotangent paths through `x`. Unbound dimensions defer to `BIND-SHAPES`, which re-propagates shapes through the IR and re-validates every rule.

Fusion. The planner groups nodes into regions under typed legality—elementwise chains fuse; a contraction admits an elementwise epilogue; a row-reduction admits prologue and epilogue; one contraction per region; reductions are barriers—and, crucially, consults a *backend-capability table*: a join is legal only if the lowering backend can emit it, so plans are lowerable by construction and each future backend capability flips one table row plus its regression. Every refusal is a typed split reason in the report (`matmul-boundary at node 2`, `barrier-boundary at node 4`). Traffic is accounted exactly—unique global reads plus writes after fusion, with broadcast discounting. For the flagship FFN block the report commits to 3,040 bytes unfused versus 2,272 fused, a 1.33× reduction computed from the model rather than asserted; the same byte number is the fusion objective, the `MEMORY` report, and the roofline denominator, making the cost model falsifiable against measured bandwidth.

Schedules and keys. Each region class has a schedule family (for GEMM: block tile, K-step, warps, stages) with a closed-form default and a bounded searched space. Selections and evidence live in a file-backed store under a content key: region signature, shape class, dtype, layout, alignment class, target architecture, *engine build* (the SHA-256 of the running toolchain binary, resolved by the engine itself from its process image), and assembler version. A schedule measured by one engine build is never silently replayed under another.

Lowering. Each region compiles to one generated PTX kernel named `REGION_`k. Four class emitters exist: flat elementwise kernels (one element per thread, masked); block-per-row reduction kernels whose layernorm, rmsnorm, and softmax bodies mirror the checked host references op-for-op through shared collective emitters; tiled GEMM with the linear bias and elementwise epilogue applied to the accumulator; and movement kernels (transpose, slice, concat, gather as index-remapped copies), with free-verdict movement dissolved into the neighboring kernel's address arithmetic instead of materializing. Analysis precedes emission and fails closed: an unsupported region emits nothing.

Whole-model execution. A multi-region model executes on device region-by-region in topological order; each materialized output stays resident in device memory and downstream regions bind producer buffers directly—model inputs are the only host uploads. The final output is compared against the host executor under a composed tolerance: summed per-region class bounds (elementwise 10^{-5} relative; reductions and matmul 10^{-4}, justified from $K \cdot 2^{-24}$ f32 accumulation plus the `ex2.approx` unit-of-last-place cost), with the host value first rounded to the f32 grid so the tolerance measures kernel error rather than representation.

5.1 Gates and Promotion

- **CERTIFY**: static legality of the whole plan.
- **GOLDEN**, with precedence: an external reference artifact (saved input/output tensors with per-dtype tolerances—the seam for porting existing models) if one exists; else the *device* model golden; else a host self-consistency oracle. Generated kernels are proven, not trusted.
- **GRADCHECK**: generated adjoints against central finite differences at tensor granularity (§6).
- **PROFILE**: measured occupancy and roofline placement; mandatory to run, non-blocking. **[TODO: populate when the roofline microbenches land.]**

`PROMOTE` admits an artifact only when certify and golden pass and gradcheck did not fail; the evidence row records exactly what ran—on the Jetson, a promoted FFN records `certify=pass|golden=device-pass:` (the golden leg's suffix names the licensed precision it was judged under). Failures render as *repair packets*: a typed failure class, the offending node or region, a reproduction pointer, and a suggested next action—designed so an agent or a person can act on the report without reading the compiler.

The gates are not ceremony; during this system's own development they caught, among others: a generated two-kernel module with a duplicated PTX header (assembles nowhere; invisible to presence-only text tests and to any host lacking `ptxas`); uniform test goldens masking index/transpose bugs (exposed by nonuniform goldens the golden gate now mandates); silently dropped device copy-backs (exposed by sentinel poisoning); and a deliberately wrong adjoint (caught by gradcheck). Each is a defect class the status-quo pipeline ships to runtime.

6 Verified Gradients and Training

Differentiation is a transform on the same IR. An adjoint registry maps each op to its VJP: elementwise ops to dedicated backward ops whose scalar references are the checked host words; matmul to transposed matmuls; bias and scale parameter-gradients to generated row-reduce and full-reduce nodes; slice and gather input-gradients to pad-scatter and scatter-add. The reverse transform walks the IR backwards, seeds the output cotangent, and *sums* fan-out contributions—a value consumed twice receives an `OP-ADD` of both cotangent paths, so a skip connection's gradient is a visible node in the IR, not a runtime convention. Save-versus-recompute decisions are recorded per node with their reasons.

Gradcheck runs at tensor granularity against the host executor: a loss $L = \sum_k s_k\, y_k$ with a varied seed (so a softmax row's gradient is not identically zero), analytic gradients read directly from backward-node buffers, central differences over perturbed inputs, and sampled corners plus midpoint per input for bounded runtime. A deliberately wrong adjoint (relu's substituted for gelu's) is caught; the detection fixture is part of the suite. Ops with no host reference report honestly as *not run*: there are no false passes by construction.

The end-to-end proof is a committed training run: a windowed MLP (`LINEAR GELU LINEAR`, 146 parameters) on a seeded synthetic regression task, Gaussian negative-log-likelihood loss over μ and $\log \sigma^2$ heads, gradients through the generated backward IR, plain SGD. From seeded initialization the batch NLL falls from 0.130 to -0.647 in 60 steps; the run is bit-deterministic—asserted with exact float equality across re-runs—and the very IR that trains is gradchecked before training. The convergence thresholds are committed as a regression: a planner or numeric change that breaks learning fails the gate like any wrong answer.

7 Error-Class Measurement

Against real Triton 3.5.1 on the same device, a six-candidate error battery (a correct kernel plus seeded authoring mistakes) shows the checked target catching the stack-discipline and composition classes at author time—located diagnostic, zero GPU—where Triton surfaces them at runtime or not at all. Authoring studies with independent generators show pass@1 = 3/3 on both targets for a fused chain: reachability is comparable; *where bugs are caught* is the difference. The earned claim is deliberately narrow: the checker types stack discipline and roles, not semantics—numeric correctness is the golden gate's job, and that division of labor is by design. **[TODO: Import the full battery table; extend candidates per the ablation matrix.]**

8 Performance

Measured today (Jetson Orin NX, sm_87, CUDA 12.6). Memory-bound: SAXPY streaming at 63 GB/s equals hand-written Triton (63.0 vs. 63.4 GB/s) at this device's achievable read/read/write bandwidth—after a vectorization fix guided by exactly the byte model above (the scalar kernel measured 42.5 GB/s; the model said memory-bound; v4 loads closed the gap). The fused `relu(`$a{\cdot}x{+}y$`)` chain reaches the same roof, produced automatically from checked words. Correctness at scale: 30 models device-golden across the four region classes and whole-model execution, including a three-region FFN with device-resident intermediate buffers.

Compute-bound: the first baseline. Square f32 GEMM $C = A \cdot B$, CUDA-event timing, same device and session:

GFLOP/s	512^3	1024^3	2048^3
generated naive tile (1 elem/thread)	55.0	55.2	54.6
generated blocked 64×64 tile	356.6	375.0	380.5
+ bk=32, vectorized shared loads	378.6	397.2	402.5
+ `cp.async` double-buffered staging	416.7	436.8	441.8
Triton, autotuned `tl.dot`	1636.1	1755.4	1890.5

The blocked schedule (shared-memory staged tiles, 4×4 register micro-tiles) delivers 6.5–7.0× over the naive tile; widening the K-tile with vectorized shared loads adds ∼6%, and asynchronous double-buffered staging another ∼10% (+16% cumulative)—every variant passing the same device golden unchanged, which is the harness doing its job: schedule work is free to iterate because correctness is re-proven per step. The remaining ≈3.9–4.3× to Triton is *arithmetic, not scheduling*: its `tl.dot` executes TF32 tensor-core instructions (measured relative error $\sim 8\times10^{-4}$ against an f32 reference, versus $\sim 10^{-6}$ for our f32 FMA kernels)—a precision our golden gate would currently *reject* under the matmul tolerance. Closing that gap is therefore gate-*licensed* precision plus `mma.sync` emission and `cp.async` pipelining—mechanical next steps of the same plan, to be reported with the same honesty. **[TODO: cp.async stages, tensor-core MMA under a licensed-precision row, persistent-vs-JIT autotune warmup, end-to-end model latency vs torch.compile.]**

8.1 Discussion: What Is and Is Not Provable

We claim no theoretical dominance over Triton's per-kernel scheduling. Its block-level dataflow analyses (coalescing, swizzling, pre-fetching, vectorization, tensor-core selection, shared-memory allocation and synchronization, asynchronous copy scheduling) and our explicit emitters plus measured schedule selection target the same hardware roofs, and both paradigms can reach them; our SAXPY history is the demonstration in miniature (a 42.5 GB/s scalar kernel, a byte model that said memory-bound, and a v4 candidate that closed to the same 63 GB/s roof Triton occupies). The difference is failure mode and recourse: heuristics miss silently—Triton's own evaluation concedes the split-K regime, and low-priority targets inherit generic configs—and the author's recourse is rewriting; here schedules are data, and the recourse is a searched candidate, selected by measurement on the deployed device and replayed by content key with zero warmup.

Why no block-level dataflow analysis. Readers comparing against Triton and PyTorch will ask where our equivalent of block-level dataflow analysis lives. The answer: that analysis exists to *rediscover* structure in an arbitrary, human-authored tile program—propagate layouts through

its def-use graph, infer where shared-memory staging pays, place barriers by hazard fixed points. Our kernel programs are *generated from the plan*, so the generator already possesses everything the analysis would infer: the schedule is explicit data (a family candidate), coalescing follows from the emitted access pattern by construction, barriers sit where the collective algorithm requires them, and the result is then *proven* by the device golden rather than trusted to an analysis. There is nothing to rediscover in a program you just generated. Our dataflow analysis operates one level up, where the status quo has none with these properties: the fusion planner's def-use analysis over the model graph (region formation, materialization, exact bytes, gradient fan-out) — the level PyTorch's Inductor occupies heuristically and without verification. The honest scaling caveat: as generated kernels grow toward tensor-core fragments and multi-stage asynchronous pipelines, pass-like machinery (register-pressure estimation, an IR optimization layer, liveness-packed shared memory) re-enters *at the point the generator needs it*; Triton's published analyses are directly reusable designs for that moment, and our roadmap tracks it explicitly.

One bounded structural argument does hold, and we state it as such. For memory-bound composites, end-to-end time is bounded below by unique bytes moved divided by achievable bandwidth. A per-kernel compiler must materialize every kernel boundary it is handed; a whole-model planner whose objective *is* unique bytes can eliminate boundaries and therefore attains a bound the per-kernel view cannot, whenever fusion removes traffic—and our byte model makes the gap exact and falsifiable per model. The honest opponent of that argument is the per-kernel paradigm, not Triton-plus-Inductor: an automatic-fusion layer above Triton occupies the same scope, and against it our residual claims are exactness (a falsifiable cost model), verification (gated artifacts), and persistence (offline-amortized tuning). Whether those structural advantages outweigh maturer per-kernel codegen on real workloads is an empirical question—precisely the one the pending measurements answer.

9 Effectiveness of Each Mechanism

Each mechanism is paired with a committed, rerunnable experiment (`docs/ablation.md` carries the full 16-row matrix with per-row file citations). Highlights, all measured on the Orin:

Golden fault injection. Four seeded kernel corruptions—wrong index arithmetic, a mis-addressed operand ($x \cdot x$ for $x \cdot y$), a dropped predication mask leaking inactive-lane ∞ into a block reduction, and a stale kernel registered for the wrong model—were each injected post-emit into otherwise-correct PTX; the golden gate caught every one (`V-FAIL`, first offending element named), and none crashed or triggered the sentinel: exactly the silent-wrong-answer class that ships in unverified pipelines.

Sentinel. A launch with the copy-back deliberately omitted throws the named readback error instead of grading stale poison as a pass.

Fusion on/off. Disabling the planner (every node its own region) on the flagship models: FFN 2272 $\rightarrow$ 3040 bytes (+25.3% traffic, 3 vs 5 regions); a linear-gelu-residual-norm block 928 $\rightarrow$ 1440 (+35.6%); a slice-into-gelu chain 192 $\rightarrow$ 320 (+40%, proving movement dissolution participates). The off-column equals the analytic per-node baseline in every row—the byte model and the planner agree by construction, and the deltas are the fusion mechanism's measured value.

Pre-existing rows are cited rather than rebuilt: the wrong-adjoint detection fixture, the seeded convergence gate, the Triton error battery, and the agent-repair-loop metrics. **[TODO: Pending rows: on-device predicted-vs-measured traffic and fused-vs-unfused latency (needs the bench harness); persistent-tune warmup vs Triton JIT (needs cad-6); EXPLAIN packet on/off A/B arm; tuned-vs-default schedule deltas.]**

9.1 Type Families

[TODO: BLOCKED on the type-families campaign merging (TFAM 9/10/12/14/15) and the cad-adt-swap adoption: checker-enforced algebraic data types (sums with MATCH and bad-tag proofs, enum and product families); adoption evidence = the CAD internals (schedule key as a product record, verdicts as sums) where the checker turns field-swap and tag-confusion into compile errors. Write from the merged implementation and its negative regressions only.]

10 Limitations

V1 caps are explicit and fail closed: one contraction per region; $K \leq 256$ and correctness-first tiles in the current GEMM (the performance tile is in progress); elementwise prologues into contractions are planned but not lowered—the backend-capability table keeps such plans unformed until the backend earns the row; broadcast operands and multi-output regions are staged behind the same mechanism; staged (transpose) movement folds materialize as copies rather than fusing. PROFILE's roofs await device microbenchmarks. The checker's scope is stack discipline and nominal roles, not program semantics. All numbers come from one device class; breadth is future work.

11 Conclusion

Habu demonstrates that a kernel target can be *checked* and a compilation pipeline can be *accountable*: typed compositions instead of unchecked authorship, exact byte accounting instead of folklore, device-proven artifacts with recorded evidence instead of emitted hope, and gradients that are generated, checked, and then trusted to train. The system is small enough to read—one self-hosted binary under 128 KB—and every claim above is a running test in its repository. What remains, deliberately unclaimed until measured, is the compute-bound contest; the machinery to run it honestly is what this paper described.

References

[1] P. Tillet, H. T. Kung, D. Cox. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. MAPL, 2019. doi:10.1145/3315508.3329973.

[2] C. H. Moore. OKAD/OKAD II: custom VLSI design tools written in (and scripting the genesis of) colorForth, used to design the MuP21, F21, c18, and GreenArrays GA144 chips. See https://colorforth.github.io/ and GreenArrays, Inc., *GA144 chip documentation*, https://www.greenarraychips.com; historical account in E. D. Rather, D. R. Colburn, C. H. Moore, "The Evolution of Forth," *History of Programming Languages—II*, ACM Press, 1996, pp. 625–670.

[3] J. Ragan-Kelley et al. Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines. PLDI, 2013.

[4] T. Chen et al. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. OSDI, 2018.

[5] N. Vasilache et al. Tensor Comprehensions: Framework-Agnostic High-Performance Machine Learning Abstractions. arXiv:1802.04730, 2018.

[6] A. Sabne. XLA: Compiling Machine Learning for Peak Performance. Google, 2020. See also OpenXLA, https://openxla.org.

[7] J. Ansel et al. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. ASPLOS, 2024.

[8] N. Rotem et al. Glow: Graph Lowering Compiler Techniques for Neural Networks. arXiv:1805.00907, 2018.

[9] C. Diggins. The Cat Programming Language: a statically typed functional stack-based language. 2007–2008, https://github.com/cdiggins/cat-language.

[10] S. Williams, A. Waterman, D. Patterson. Roofline: An Insightful Visual Performance Model for Multicore Architectures. CACM 52(4), 2009.